

ДЗ_7_АЙКОС

Комкова Полина Дмитриевна БПИ244

Формулировка ДЗ

Разработать программу использующую для работы с **текстовыми** файлами **только системные вызовы**. Программа на языке C должна прочитать, используя буфер, размер которого **превышает** читаемые файлы и записать прочитанный файл в файл с другим именем. Имена файлов для чтения и записи задавать с использованием **аргументов командной строки**.

Вместо большого буфера использовать для работы с файлами **буфер ограниченного размера равный 32 байт**, требующий циклического использования как для чтения, так и для записи.

Читать и переписывать не только текстовые, но и исполняемые файлы (**текстовые и бинарные, содержащие признак исполнимости**), включая скрипты, которые сохраняют режим доступа исходных файлов, обеспечивающий их запуск. При этом обычные текстовые файлы запускаться не должны. Для них режим доступа должен оставаться прежним.

Исходный код

```
#include <sys/types.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <errno.h>

int buf_size = 32; // Разбер буфера для чтения/записи

int main(int argc, char* argv[]) {
    int fd_src = 0; // Дескриптор исходного файла
    int fd_dst = 0; // Дескриптор целевого файл
    struct stat src_stat; // Структура информации об исходном файле
```

```
ssize_t read_bytes = 0; // Количество прочитанных байтов
char buffer[buf_size]; // Буфер чтения/записи данных
int rval = 0; // Для проверки прав доступа

// Проверка аргументов командной строки
if(argc != 3) {
    printf("Как нужно: %s исходный_файл целевой_файл\n", argv[0]);
    exit(-1);
}

// Открываем исходный файл только для чтения
if((fd_src = open(argv[1], O_RDONLY)) < 0) {
    printf("Нельзя открыть исходный файл\n");
    exit(-1);
}

// Получаем информацию об исходном файле для прав доступа
if(fstat(fd_src, &src_stat) < 0) {
    printf("Нельзя получить информацию об исходном файле\n");
    close(fd_src);
    exit(-1);
}

// Создаем файл с базовыми правами
if((fd_dst = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0666)) < 0) {
    printf("Нельзя создать целевой файл\n");
    close(fd_src);
    exit(-1);
}

// Циклическое чтение и запись с буфером в 32 байта
do {
    read_bytes = read(fd_src, buffer, buf_size);
    if(read_bytes == -1) {
        printf("Ошибка чтения из исходного файла\n");
        close(fd_src);
        close(fd_dst);
        exit(-1);
    }

    if(read_bytes > 0) {
        if(write(fd_dst, buffer, read_bytes) != read_bytes) {
            printf("Ошибка записи в целевой файл\n");
            close(fd_src);
            close(fd_dst);
            exit(-1);
        }
    }
}
```

```

    }
}
} while(read_bytes == buf_size); // Продолжаем, пока читаем полные блоки

// Закрываем файлы
if(close(fd_src) < 0) {
    printf("Ошибка закрытия исходного файла\n");
}
if(close(fd_dst) < 0) {
    printf("Ошибка закрытия целевого файла\n");
}

// Проверка, является ли исходный файл исполняемым
rval = access(argv[1], X_OK);
if (rval == 0) {
    // Если файл исполняемый, то устанавливаем оригинальные права доступа
    if(chmod(argv[2], src_stat.st_mode) < 0) {
        printf("Нельзя установить права доступа для исполняемого файла\n");
    } else {
        printf("Файл %s является исполняемым – права доступа сохранены\n",
argv[1]);
    }
} else {
    // Иначе если обычный текстовый файл, то убираем все биты исполнения
    mode_t text_mode = src_stat.st_mode & ~(S_IXUSR | S_IXGRP | S_IXOTH);
    if(chmod(argv[2], text_mode) < 0) {
        printf("Нельзя установить права доступа для текстового файла\n");
    } else {
        printf("Файл %s является текстовым – биты исполнения удалены\n",
argv[1]);
    }
}

printf("Файл успешно скопирован с учетом прав доступа\n");
return 0;
}

```

Идея

- **Циклическое чтение/запись** - файл обрабатывается частями по 32 байта через буфер ограниченного размера
- **Автоматическое определение типа файла** - программа анализирует права доступа исходного файла

- **Управление правами:**
 - Исполняемые файлы сохраняют все права доступа (биты +x)
 - Текстовые файлы лишаются прав на исполнение
- **Обработка ошибок** - все системные вызовы проверяются на корректность выполнения

Пояснения

Все вроде очевка, кроме, может быть, проверки на исполняемость файла и смены режимов доступа

```
// Проверка, является ли исходный файл исполняемым
rval = access(argv[1], X_OK);
if (rval == 0) {
    // Если файл исполняемый, то устанавливаем оригинальные права доступа
    if(chmod(argv[2], src_stat.st_mode) < 0) {
        printf("Нельзя установить права доступа для исполняемого файла\n");
    } else {
        printf("Файл %s является исполняемым – права доступа сохранены\n",
argv[1]);
    }
} else {
    // Иначе если обычный текстовый файл, то убираем все биты исполнения
    mode_t text_mode = src_stat.st_mode & ~(S_IXUSR | S_IXGRP | S_IXOTH);
    if(chmod(argv[2], text_mode) < 0) {
        printf("Нельзя установить права доступа для текстового файла\n");
    } else {
        printf("Файл %s является текстовым – биты исполнения удалены\n",
argv[1]);
    }
}
```

- `chmod(argv[2], src_stat.st_mode)` устанавливает для целевого файла ТОЧНО ТАКИЕ ЖЕ права, как у исходного файла
- `S_IXUSR | S_IXGRP | S_IXOTH` - побитовое ИЛИ создает маску всех битов исполнения:
 - `S_IXUSR` - исполнение для владельца
 - `S_IXGRP` - исполнение для группы
 - `S_IXOTH` - исполнение для остальных
- `~` - побитовое НЕ переворачивает маску (теперь она для снятия битов)

- `src_stat.st_mode & ~(...)` - побитовое И снимает биты исполнения, сохраняя остальные права

Пример использования

```
polia@LAPTOP-KJ14OKRT: ~/AKOC/DZ_7
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ gcc -o main DZ_7.c
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ./main text_file.txt text_copy.txt
Файл text_file.txt является текстовым - биты исполнения удалены
Файл успешно скопирован с учетом прав доступа
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ls -l text_file.txt text_copy.txt
-rw-r--r-- 1 polia polia 50 Nov 17 18:38 text_copy.txt
-rw-r--r-- 1 polia polia 50 Nov 17 18:33 text_file.txt
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ./main script.sh script_copy.sh
Файл script.sh является исполняемым - права доступа сохранены
Файл успешно скопирован с учетом прав доступа
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ls -l script.sh script_copy.sh
-rwxr-xr-x 1 polia polia 42 Nov 17 18:36 script.sh
-rwxr-xr-x 1 polia polia 42 Nov 17 18:39 script_copy.sh
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ./script_copy.sh
Привет, мир!
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ./text_copy.txt
-bash: ./text_copy.txt: Permission denied
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$ ./main qwerty.txt output.txt
Нельзя открыть исходный файл
polia@LAPTOP-KJ14OKRT:~/AKOC/DZ_7$
```

- `-rw-r--r--` - обычный текстовый файл, который владелец может читать и изменять, а все остальные - только читать
- `-rwxr-xr-x` - исполняемый файл, который:
 - Владелец может читать, изменять и запускать
 - Все остальные пользователи могут читать и запускать

Структура файла

project/	
├─ DZ_7.c	# Исходный код
├─ text_file.txt	# Тестовый текстовый файл
├─ script.sh	# Тестовый исполняемый скрипт
└─ README.md	# Описание программы и тестов

Компиляция

```
gcc -o main DZ_7.c
```

Тестирование

```
1) ./main text_file.txt copy_text.txt  
2) ./main script.sh copy_script.sh
```

Функционал

- Копирование файлов с буфером 32 байта
- Сохранение прав доступа для исполняемых файлов
- Автоматическое определение типа файла
- Обработка ошибок

Соответствие требованиям

-  **6 баллов:** Системные вызовы, аргументы командной строки
-  **+1 балл:** Буфер 32 байта, циклическое чтение/запись
-  **+1 балл:** Обработка исполняемых файлов, сохранение прав доступа